DAY-BY-DAY BUILD CHECKLIST

Vietnamese Travel Phrasebook App — Zero to App Store

V1: No voice recording. Beautiful phrasebook with native audio, save-to-favorites, offline access.

Working estimate: 20 working days at 12 hours/day Scope: 10 scenarios, ~60 phrases, audio playback, favorites, offline, App Store ready

---

BEFORE DAY 1: THE STUFF NOBODY TELLS A BEGINNER

Things you need to understand before writing a line of code:

What is a "terminal"? It's the text-based command line on your Mac. Open it: Cmd+Space → type "Terminal" → Enter. Everything in this guide that starts with `$` means "type this into your terminal."

What is Node.js? It's the engine that runs JavaScript outside a browser. Expo needs it. You install it once.

What is npm? It's like an app store for code packages. It comes with Node.js.

What is Git? It's version control — think of it as infinite undo for your entire project. GitHub is the cloud backup.

What is VS Code? It's the text editor where you'll review code. Think of it as Microsoft Word for code.

What is Expo? It's a framework that lets you build iPhone apps using JavaScript instead of Swift. It handles 90% of the hard iPhone stuff for you.

What is Expo Go? It's an app on your physical iPhone that lets you preview your app during development without building a full .ipa file. Much faster than building each time.

What is EAS Build? It's Expo's cloud service that compiles your JavaScript into an actual iPhone app binary. You need this for TestFlight and the App Store.

What is TestFlight? It's Apple's system for distributing beta apps to testers before they go on the App Store.

What is Claude Code? It's an AI coding agent that runs in your terminal. You describe what you want in English, and it reads your files, writes code, and runs commands. This is your primary building tool.

What is NativeWind? It's a way to style React Native apps using Tailwind CSS class names (like `bg-blue-500`, `text-xl`, `rounded-lg`). It's the fastest way to make your app look good.

What is AsyncStorage? It's a simple key-value storage on the user's phone. Think of it like a tiny local database. You'll use it for saved favorites and settings.

PRE-BUILD: THINGS TO SET UP ON YOUR MAC (Do this the night before Day 1)

Checklist: Mac Setup

- **Install Node.js** (version 20 or later)
  - Go to https://nodejs.org → Download the LTS version → Install
  - Verify: open Terminal, type `node --version` — should show v20.x or higher
- **Install Git**
  - It's probably already installed. Type `git --version` in Terminal
  - If not: install Xcode Command Line Tools by typing `xcode-select --install`
- **Create a GitHub account** (if you don't have one)
  - Go to https://github.com → Sign up
  - This is where your code lives in the cloud
- **Install VS Code**
  - Go to https://code.visualstudio.com → Download → Install
  - This is where you'll review code that Claude writes
- **Install Claude Code**
  - In Terminal: `npm install -g @anthropic-ai/claude-code`

    - Then run `claude` in Terminal and sign in with your Claude Max account
    - Verify: type `claude --version`
- **Install Expo Go on your iPhone**
    - Go to the App Store on your phone → Search "Expo Go" → Install
    - This lets you preview your app on your real phone during development
- **Create an Apple Developer Account**
    - Go to https://developer.apple.com/programs/enroll/
    - Enroll as an Individual ($99/year)
    - **WARNING: Processing can take 24-48 hours.** Do this ASAP.
    - You need this for TestFlight and the App Store. You cannot skip this.
- **Create an Expo account**
    - Go to https://expo.dev → Sign up
    - You'll need this for EAS Build later
- **Create an OpenAI API account** (separate from ChatGPT)
    - Go to https://platform.openai.com → Sign up
    - Add $10 of credits (this will last months for your use)
    - Create an API key: Settings → API Keys → Create new key → Copy and save it somewhere safe
- **Install the Expo CLI**
    - In Terminal: `npm install -g eas-cli`
    - Then login: `eas login`
- **Ensure your Mac and iPhone are on the same WiFi network**
    - This is required for Expo Go to connect to your development server

---

DESIGN SYSTEM (Decided in advance — do NOT spend days on this)

You asked about design. Here's the deal: you are not a designer, and you should not try to become one during this project. Instead, you're going to use a pre-defined design system that Claude Code will implement consistently. This is decided now and doesn't change.

Color Palette

```
Primary:        #FF6B35  (warm orange — energetic, travel-friendly)
Primary Dark:   #E55A2B  (pressed state)
Background:     #FAFAF8  (warm off-white — easier on eyes than pure white)
Surface:        #FFFFFF  (cards, modals)
Text Primary:   #1A1A2E  (near-black, high contrast)
Text Secondary: #6B7280  (gray for subtitles)
Vietnamese:     #1A1A2E  (Vietnamese text — always dark and large)
Romanized:      #FF6B35  (pronunciation guide — primary color, stands out)
English:        #6B7280  (meaning — gray, secondary importance)
Success:        #10B981  (correct/completed)
Border:         #E5E7EB  (light gray borders)
Tab Active:     #FF6B35  (active tab)
Tab Inactive:   #9CA3AF  (inactive tab)
```

Typography Hierarchy

```
Vietnamese phrase:    24px, bold, #1A1A2E    (the star of every card)
Romanized guide:     18px, medium, #FF6B35  (what the user actually reads to spea
English meaning:     14px, regular, #6B7280 (reference, not primary)
Screen title:        28px, bold, #1A1A2E
Section header:      18px, semibold, #1A1A2E
Body text:           16px, regular, #374151
Caption/metadata:    12px, regular, #9CA3AF
```

Component Design Rules

- Card radius: `rounded-2xl` (16px) — modern, friendly
- Button radius: `rounded-xl` (12px)
- Card shadow: `shadow-sm` — subtle, not heavy
- Spacing between cards: 12px
- Screen padding: 20px horizontal
- Play button: circle, 48x48, primary orange background, white play icon

- Save button: heart outline (unsaved) / filled heart (saved), primary orange
- Bottom tab bar: 2 tabs only — “Learn” (home icon) and “Saved” (heart icon)
- No hamburger menus. No drawers. Settings is accessed from a gear icon in the header.

App Icon Direction

Describe to Higgsfield AI: “A minimal, modern app icon with a warm orange speech bubble containing Vietnamese text on a cream/white background. No text, no faces, no flags. Clean, geometric, approachable.”

Screen Layouts (What Each Screen Looks Like)

Home Screen

```
┌──────────────────────────────┐
│  [App Name]          ⚙️ gear  │  ← header
│                              │
│  Travel Vietnamese           │  ← subtitle: "Tap a situation to start"
│                              │
│  ┌──────────┐ ┌──────────┐   │
│  │ ☕       │ │ 🍜       │   ← 2-column grid of scenario cards
│  │Coffee Shop│ │Street Food│   with emoji + name + "X phrases"
│  │ 6 phrases│ │ 7 phrases│   + completion indicator
│  └──────────┘ └──────────┘   │
│  ┌──────────┐ ┌──────────┐   │
│  │ 🚕       │ │ 💰       │
│  │Grab / Taxi│ │Ask Price │
│  │ 6 phrases│ │ 5 phrases│
│  └──────────┘ └──────────┘   │
│  ... (scrollable)            │
│                              │
│  ────────────────────────    │
│  🏠 Learn          ❤️ Saved  │  ← bottom tabs
└──────────────────────────────┘
```

Scenario/Phrase List Screen

```
┌──────────────────────────────┐
│  ← Back    Coffee Shop       │  ← header with back button
│                              │
│  "Order like a local" ☕      │  ← scenario intro, one line
```

```
│                                │
│  ┌──────────────────────────┐  │
│  │ Cho tôi một cà phê sữa   │  │  ← Vietnamese (large, bold)
│  │ cho toy moht kah feh sua │  │   ← romanized (orange, medium)
│  │ One iced milk coffee     │  │   ← English (gray, small)
│  │                    🔊 ❤️ │  │   ← play button + save button
│  └──────────────────────────┘  │
│  ┌──────────────────────────┐  │
│  │ Bao nhiêu tiền?          │  │
│  │ bow nyew tee-en          │  │
│  │ How much?                │  │
│  │                    🔊 ❤️ │  │
│  └──────────────────────────┘  │
│  ... (scrollable)              │
│                                │
│  ──────────────────────────    │
│  🏠 Learn          ❤️ Saved    │
└────────────────────────────────┘
```

Favorites Screen

```
┌────────────────────────────────┐
│  Saved Phrases                 │  ← header
│                                │
│  Quick access — works offline  │  ← subtitle
│                                │
│  [same phrase card layout]     │  ← but with filled hearts
│  [same phrase card layout]     │     and "from: Coffee Shop" label
│  [same phrase card layout]     │
│                                │
│  (empty state if no saves:     │
│   "Save phrases from any       │
│    lesson for quick access     │
│    — even offline!")           │
│                                │
│  ──────────────────────────    │
│  🏠 Learn          ❤️ Saved    │
└────────────────────────────────┘
```

Settings Screen

```
┌────────────────────────────────┐
│  ← Back        Settings        │
│                                │
```

```
│  About this app              │  ← simple grouped list
│  Privacy Policy              │  ← opens URL
│  Terms of Use                │  ← opens URL
│  Send Feedback               │  ← opens email compose
│  Rate this App               │  ← opens App Store (later)
│  Version 1.0.0               │  ← small text at bottom
└──────────────────────────────┘
```

THE 20-DAY BUILD PLAN

DAY 1: Environment, Repo, Project Scaffold

Hours: 12 | Objective: Working Expo app running on your iPhone

Morning Block (6 hours): Setup Everything

Step 1: Verify all pre-build items are done (30 min) Open Terminal. Run each of these and confirm they work:

```
node --version      # Should show v20+
git --version       # Should show a version
npm --version       # Should show a version
claude --version    # Should show Claude Code version
eas --version       # Should show EAS CLI version
```

Step 2: Create your project folder and GitHub repo (30 min)

Open Claude Code in Terminal:

```
mkdir ~/Projects
cd ~/Projects
claude
```

Prompt for Claude Code:

```
I'm starting a brand new project from scratch. Help me:

1. Create a new GitHub repository called "viet-travel-phrases"
   (initialize it on GitHub first, then clone locally)
2. Walk me through the git setup step by step since I'm new to this
```

```
I have git installed and a GitHub account. My GitHub username is [YOUR_USERNAME].
Talk me through each step. Don't skip anything.
```

Step 3: Create the Expo project (2 hours)

Prompt for Claude Code:

```
Create a new Expo project inside my viet-travel-phrases repo with these exact spe

Tech:
- Expo SDK (latest stable)
- TypeScript strict mode
- Expo Router with file-based routing
- NativeWind (Tailwind CSS for React Native) with Tailwind v3

Project structure:
app/
  (tabs)/
    _layout.tsx          # Tab navigator with 2 tabs: Learn and Saved
    index.tsx            # Home screen - scenario grid
    saved.tsx            # Saved/favorites phrases
  scenario/
    [id].tsx             # Phrase list for a specific scenario
  settings.tsx           # Settings screen
components/
  PhraseCard.tsx         # Reusable phrase card component
  ScenarioCard.tsx       # Reusable scenario tile for home grid
  AudioPlayButton.tsx    # Play button that handles audio
  SaveButton.tsx         # Heart toggle for saving phrases
content/
  scenarios.ts           # TypeScript file exporting all scenario data
  types.ts               # TypeScript interfaces for Scenario, Phrase
assets/
  audio/                 # Will hold .mp3 files later (empty for now)

For now, use placeholder data: 2 scenarios with 3 phrases each.
Include proper TypeScript types for everything.

NativeWind setup: follow the official docs exactly. Include:
- tailwind.config.js with content paths
- global.css with @tailwind directives
- metro.config.js with withNativeWind
- babel.config.js with nativewind/babel preset
- nativewind-env.d.ts for TypeScript
```

```
Run `npx expo start` at the end to verify everything works.
Show me any errors and fix them.
```

What will go wrong (and that's OK):

- NativeWind setup has several config files that must be exactly right. If the app shows unstyled content, the metro.config.js or babel.config.js is wrong. Tell Claude Code the exact error.
- If Expo Go can't connect, make sure your phone and Mac are on the same WiFi.

Step 4: Get it running on your physical iPhone (1 hour)

After Claude Code runs `npx expo start`, you'll see a QR code in your terminal.

1. Open your iPhone camera
2. Point it at the QR code
3. Tap the banner that says "Open in Expo Go"
4. The app should load on your phone

If it doesn't connect, tell Claude Code:

```
Expo Go on my iPhone can't connect to the dev server.
My Mac IP is [run `ifconfig | grep inet` to find it].
My phone is on the same WiFi network called [NETWORK_NAME].
Help me debug this.
```

Afternoon Block (6 hours): Design System + CLAUDE.md

Step 5: Implement the design system (3 hours)

Prompt for Claude Code:

```
Now let's implement the design system for our app. Update the following:

1. tailwind.config.js - add custom colors:
   primary: '#FF6B35'
   primaryDark: '#E55A2B'
   background: '#FAFAF8'
   surface: '#FFFFFF'
   textPrimary: '#1A1A2E'
   textSecondary: '#6B7280'
   vietnamese: '#1A1A2E'
   romanized: '#FF6B35'
```

```
   english: '#6B7280'
   success: '#10B981'
   border: '#E5E7EB'

2. Create components/ui/ThemedText.tsx with variants:
   - vietnamese: 24px bold textPrimary
   - romanized: 18px medium romanized (orange)
   - english: 14px regular english (gray)
   - title: 28px bold textPrimary
   - subtitle: 18px semibold textPrimary
   - body: 16px regular gray-700
   - caption: 12px regular gray-400

3. Update the home screen (app/(tabs)/index.tsx) to show a 2-column grid
   of scenario cards using the placeholder data. Each card shows:
   - An emoji
   - Scenario name
   - "X phrases" count
   - Rounded corners (rounded-2xl), subtle shadow, white background

4. Style the tab bar:
   - 2 tabs: "Learn" (home icon) and "Saved" (heart icon)
   - Active color: #FF6B35, inactive: #9CA3AF
   - Clean, minimal tab bar styling

Use NativeWind classes throughout. No StyleSheet objects.
After making changes, verify the app renders correctly.
```

Step 6: Create the CLAUDE.md (1 hour)

Prompt for Claude Code:

```
Create a CLAUDE.md file at the project root with this exact content,
then also create a .cursorrules file with the same content (some tools read
different files):

# Vietnamese Travel Phrasebook App

## What This Is
iPhone-first phrasebook app for English-speaking travelers in Vietnam.
10 travel scenarios, ~60 phrases with pre-generated native audio.
Tap to play, save to favorites, works offline.

## V1 Scope — DO NOT exceed this
- 10 scenario screens with phrase cards
- Audio playback (pre-generated, bundled in app)
```

```
- Save/unsave phrases to favorites
- Favorites screen with offline audio
- Settings with privacy policy link
- NO voice recording, NO transcription, NO AI API calls at runtime
- NO subscriptions, NO accounts, NO login
- NO gamification, NO streaks, NO progress tracking beyond completion

## Tech Stack
- Expo SDK latest, Expo Router, TypeScript strict
- NativeWind (Tailwind CSS v3) for ALL styling
- expo-av for audio playback
- @react-native-async-storage/async-storage for favorites
- No backend needed for V1 (all content bundled)

## Design System
- Primary: #FF6B35 (warm orange)
- Background: #FAFAF8 (warm off-white)
- Vietnamese text: 24px bold #1A1A2E
- Romanized: 18px medium #FF6B35
- English: 14px regular #6B7280
- Cards: rounded-2xl, shadow-sm, white background
- Buttons: rounded-xl

## Code Rules
- Functional components only, named exports
- NativeWind classes only — never use StyleSheet
- Max 150 lines per file, split if longer
- Every component handles: loading, error, empty states
- TypeScript strict — no `any` types
- Keep it simple. This is an MVP.

## Content Rules
- Vietnamese: Southern dialect preferred
- Romanized: intuitive English phonetics, NOT IPA
- All content is static TypeScript, not fetched from a server
```

Step 7: First git commit (30 min)

Prompt for Claude Code:

```
Stage all files and make our first git commit with message:
"Day 1: Project scaffold with Expo Router, NativeWind, and design system"
Then push to GitHub.
Walk me through each git step and explain what's happening.
```

Step 8: Document what happened (30 min)

Create a file called `BUILD_LOG.md` in your project. Write:

- What worked
- What broke and how you fixed it
- How long each step actually took
- Any open questions

✅ Day 1 Success Criteria

- App runs on your physical iPhone via Expo Go
- Home screen shows a grid of placeholder scenario cards with the design system colors
- Two tab bar icons work (Learn and Saved)
- NativeWind styling is working (you see orange and the correct fonts)
- Code is committed and pushed to GitHub
- CLAUDE.md is at the project root

⚠️ Day 1 Biggest Risk

NativeWind configuration is finicky. If styles aren't applying, the #1 fix is:

```
npx expo start --clear
```

This clears the Metro cache. If that doesn't work, verify metro.config.js, babel.config.js, and global.css are all set up exactly per NativeWind docs.

DAY 2: Content Creation — All 10 Scenarios + Phrases

Hours: 12 | Objective: All 60+ phrases written, validated structure

Morning Block (6 hours): Generate All Content

You're spending an entire day on content because **content IS your product.** A phrasebook with bad phrases is worse than no phrasebook at all.

Step 1: Generate content using Claude Chat (not Claude Code)

Open claude.ai in your browser. Create a new Project called "Vietnamese Travel App Content."

Upload your CLAUDE.md to the project.

For EACH of the 10 scenarios, use this prompt. Do them one at a time, review each before moving to the next:

Prompt for Claude Chat (repeat 10 times, changing the scenario):

```
You are a Vietnamese language expert who lives in Vietnam and teaches
practical Vietnamese to English-speaking tourists every day.

Generate content for this travel scenario: COFFEE SHOP

Requirements:
- 6-8 phrases maximum
- Southern Vietnamese dialect (Saigon/HCMC style, which is also understood in Da
- Only phrases a tourist would ACTUALLY use in this exact situation
- Order from most useful to least useful
- Keep phrases short — under 8 Vietnamese words each

For each phrase, provide:
1. vietnamese: the Vietnamese text with all diacritical marks (tones are critical
2. romanized: an intuitive English phonetic guide. NOT IPA.
   Write it so a monolingual English speaker would get close to the right sound.
   Use hyphens to separate syllables. Add CAPS for stressed parts.
   Example: "Cảm ơn" → "kahm UHN"
3. english: the English meaning
4. context: a one-sentence explanation of WHEN to use this phrase
5. emoji: one emoji that represents this phrase

Output as a JSON array so I can copy it directly into my app code.

Do NOT include:
- Grammar explanations
- Formal/literary Vietnamese
- Phrases a tourist would never actually say
- Phrases that require a response the tourist can't understand
```

Step 2: Organize into the scenarios.ts file (2 hours)

After generating all 10 scenarios, compile them.

Prompt for Claude Code:

```
I have all my lesson content ready. Update content/scenarios.ts and content/types

Here are the TypeScript interfaces I want:
```

```
interface Phrase {
  id: string;           // e.g., "coffee-1"
  vietnamese: string;   // with all diacritical marks
  romanized: string;    // English phonetic guide
  english: string;      // meaning
  context: string;      // when to use this
  emoji: string;        // single emoji
  audioFile: string;    // filename like "coffee-1.mp3"
}

interface Scenario {
  id: string;           // e.g., "coffee-shop"
  name: string;         // "Coffee Shop"
  emoji: string;        // "☕"
  description: string;  // "Order like a local"
  phrases: Phrase[];
}

Now here is all the content I generated:

[PASTE ALL YOUR JSON CONTENT HERE]

Create the complete scenarios.ts file with all data as a typed export.
Make sure every Vietnamese character and tone mark is preserved exactly.
Generate the audioFile names following the pattern: {scenarioId}-{number}.mp3
```

Afternoon Block (6 hours): Content Review + Phrase Card UI

Step 3: Get a native speaker review (2 hours)

This is THE most important step in your entire project. Find a Vietnamese friend, pay a tutor on italki $15-20 for an hour, or find someone at a Da Nang café.

Show them all 60+ phrases. For each one, ask:

- "Would a Vietnamese person understand this?"
- "Is this the natural way to say it?"
- "Would you change anything?"

Fix every issue they identify.

Step 4: Build the phrase card component (3 hours)

Prompt for Claude Code:

```
Build the PhraseCard component at components/PhraseCard.tsx.

It receives a Phrase object and displays:
- Vietnamese text: large (text-2xl), bold, dark color
- Romanized pronunciation: medium (text-lg), orange primary color
- English meaning: small (text-sm), gray
- Context note: extra small (text-xs), gray-400, italic
- A play button (🔊) on the right side — circular, orange bg, white icon
  - For now, the play button just logs "play: {audioFile}" to console
  - We'll add real audio later
- A save/heart button — outline when unsaved, filled when saved
  - Use AsyncStorage to persist saved state
  - Key format: "saved-{phraseId}"

Layout:
- White card with rounded-2xl, shadow-sm, p-4
- Vietnamese/romanized/english stacked on the left
- Play + save buttons on the right
- 12px gap between cards

Also build SaveButton.tsx that handles the AsyncStorage toggle.
Use @react-native-async-storage/async-storage (install it if not already).

Make sure to handle the initial load state (checking if a phrase is saved).
```

Step 5: Wire phrase cards into the scenario screen (1 hour)

Prompt for Claude Code:

```
Update app/scenario/[id].tsx to:
1. Read the scenario ID from the route params
2. Look up the scenario from our scenarios.ts data
3. Display a header with the scenario emoji, name, and description
4. Show a scrollable list of PhraseCard components for each phrase
5. Add a back button that returns to the home screen

Use a FlatList for the phrase list (better performance than mapping).
Add the Expo Router back button in the header.
Style everything with NativeWind using our design system.
```

✅ Day 2 Success Criteria

- All 10 scenarios with all phrases are in scenarios.ts
- At least one native Vietnamese speaker has reviewed the content

- PhraseCard component renders correctly with Vietnamese text (including tone marks)
- Save/unsave hearts work and persist between app restarts
- Tapping a scenario on the home screen navigates to the phrase list
- The app looks clean and matches the design system

⚠️ Day 2 Biggest Risk

Vietnamese diacritical marks (tone marks) getting corrupted in the code file. If you see `â` or `Ã` instead of proper Vietnamese characters, your file encoding is wrong. Tell Claude Code: "The Vietnamese characters are corrupted. Make sure scenarios.ts is saved as UTF-8 encoding."

DAY 3: Audio Generation + Playback Integration

Hours: 12 | Objective: Every phrase plays Vietnamese audio on your iPhone

Morning Block (6 hours): Generate All Audio Files

Step 1: Create a TTS generation script (1 hour)

Prompt for Claude Code:

```
Create a Node.js script at scripts/generate-audio.ts that:

1. Reads all phrases from content/scenarios.ts
2. For each phrase, calls the OpenAI TTS API to generate Vietnamese audio:
   - Model: gpt-4o-mini-tts
   - Voice: "coral" (test this — we may change it)
   - Input: the Vietnamese text from each phrase
   - Instructions: "Speak slowly and clearly in Vietnamese.
     This is for language learners. Pause briefly between words."
   - Format: mp3
3. Saves each audio file to assets/audio/{scenarioId}-{number}.mp3
4. Logs progress: "Generated 1/60: coffee-1.mp3"
5. Adds a 1-second delay between API calls to avoid rate limiting
6. Skips files that already exist (so we can re-run without regenerating everythi

The script needs my OpenAI API key. Read it from an environment variable
OPENAI_API_KEY. Create a .env file for it (and add .env to .gitignore).

Install any needed packages (openai, dotenv, tsx for running TypeScript).
```

Step 2: Run the script and generate all audio (2 hours)

```
# Set your API key
echo "OPENAI_API_KEY=sk-your-key-here" > .env

# Run the generation script
npx tsx scripts/generate-audio.ts
```

This will take 15-30 minutes and cost roughly $1-2 in API calls.

Step 3: Listen to every single audio file (2 hours)

THIS IS NOT OPTIONAL. Open each .mp3 file on your Mac (Finder → double-click) and listen.

Check for:

- Does it sound like clear Vietnamese?
- Is the speed appropriate for a learner?
- Are the tones correct? (Ask your Vietnamese friend to spot-check 10 of them)
- Is the voice pleasant?

If "coral" doesn't sound good for Vietnamese, regenerate with a different voice.

Prompt for Claude Code (if you need to try a different voice):

```
The TTS voice "coral" doesn't sound natural for Vietnamese.
Update the generate-audio script to use voice "nova" instead.
Delete all existing audio files and regenerate them.
```

Step 4: Test multiple voices (1 hour)

Generate the SAME phrase with all available voices and compare:

```
Create a test script at scripts/test-voices.ts that generates
the phrase "Cho tôi một cà phê sữa đá" using all 11 OpenAI TTS voices:
alloy, ash, ballad, coral, echo, fable, nova, onyx, sage, shimmer, verse

Save them as assets/audio/test-{voice}.mp3 so I can compare.
```

Listen to all 11 and pick the best one for Vietnamese.

Afternoon Block (6 hours): Wire Audio Into the App

Step 5: Implement the AudioPlayButton (3 hours)

Prompt for Claude Code:

```
Implement the AudioPlayButton component at components/AudioPlayButton.tsx.

Requirements:
- Uses expo-av (install it: npx expo install expo-av)
- Receives an audioFile prop (filename like "coffee-1.mp3")
- Loads the audio from the bundled assets: require(`../../assets/audio/${audioFil
  - IMPORTANT: React Native require() doesn't support dynamic paths.
    We need to create an audio registry file that maps filenames to
    require() calls for each file. Create assets/audio/registry.ts
    that exports a Record<string, number> mapping each filename to its
    require() statement.
- Shows a play/stop icon (circular button, 48x48, orange bg)
- While playing: shows a stop icon and the button pulses slightly
- When audio finishes: returns to play icon
- Only one audio can play at a time — playing a new phrase stops the previous one
- Handle errors gracefully (missing file → show a small "!" icon)

Also update PhraseCard to use the real AudioPlayButton instead of the placeholder

CRITICAL iOS NOTE: Call Audio.setAudioModeAsync({ playsInSilentMode: true })
in the app's root layout so audio plays even when the phone is on silent.
```

Step 6: Create the audio registry (1 hour)

Prompt for Claude Code:

```
The audio registry needs to map every audio filename to a require() call
because React Native can't do dynamic requires.

Create a script at scripts/generate-audio-registry.ts that:
1. Reads all .mp3 files in assets/audio/
2. Generates assets/audio/registry.ts with content like:

const audioRegistry: Record<string, any> = {
  "coffee-1.mp3": require("./coffee-1.mp3"),
  "coffee-2.mp3": require("./coffee-2.mp3"),
  // ... for all files
};
export default audioRegistry;

Run this script now and verify the registry file is correct.
```

Step 7: Test audio on your iPhone (1 hour)

Shake your phone or press Cmd+R in Terminal to reload the app. Test:

- Tap play on 5 different phrases across 2 scenarios
- Test with phone on SILENT mode (audio should still play)
- Test playing one phrase while another is playing (old one should stop)
- Test navigating away while audio is playing (should stop)

Step 8: Fix any audio issues (1 hour)

Common problems and what to tell Claude Code:

- "Audio doesn't play on my iPhone" → Probably need `playsInSilentMode: true`
- "Audio plays twice when I tap quickly" → Need to debounce the play button
- "Audio keeps playing when I leave the screen" → Need cleanup in useEffect
- "I get a red error screen about require()" → The audio registry has a wrong path

✅ Day 3 Success Criteria

- All 60+ audio files are generated and in assets/audio/
- Audio plays on your physical iPhone for every phrase
- Audio plays when phone is on silent mode
- Only one audio plays at a time
- Audio stops when navigating away from the screen
- You (and ideally a Vietnamese friend) have listened to at least 20 audio files and confirmed they sound correct

⚠️ Day 3 Biggest Risk

The require() dynamic path issue. React Native CANNOT dynamically require files like `require(\ ./${filename}`)`. The audio registry pattern is the standard solution. If Claude Code tries to use dynamic requires, tell it: "React Native doesn't support dynamic require. Use the registry pattern."

DAY 4: Favorites Screen + Offline Mode + Home Screen Polish

Hours: 12 | Objective: Complete favorites flow with offline audio

Morning Block (6 hours)

Step 1: Build the Favorites screen (3 hours)

Prompt for Claude Code:

```
Build the Saved/Favorites screen at app/(tabs)/saved.tsx.

Requirements:
- On mount, read all saved phrase IDs from AsyncStorage
- Look up full phrase data from scenarios.ts
- Display saved phrases as PhraseCard components (same as scenario screen)
- Add a small label under each card showing which scenario it came from
  (e.g., "from Coffee Shop")
- Implement an empty state with:
  - A large heart outline icon
  - Text: "No saved phrases yet"
  - Subtext: "Tap ❤️ on any phrase to save it for quick access — even offline!"
- When a phrase is unsaved from this screen, remove it with a nice
  fade-out animation using react-native-reanimated (already installed with Native
- Phrases should be sorted by most recently saved first
- Pull-to-refresh to reload saved phrases

CRITICAL: Audio must work on this screen even when the phone has no internet.
Since our audio files are bundled in the app (not fetched from a server),
this should work automatically. Verify this by turning off WiFi on your phone
and testing audio playback.
```

Step 2: Polish the home screen (3 hours)

Prompt for Claude Code:

```
Polish the home screen at app/(tabs)/index.tsx.

Current state: basic grid of scenario cards.

Updates needed:
1. Add a header area:
   - App name (we'll decide the name later, use "Xin chào!" for now)
   - Subtitle: "Travel Vietnamese for everyday situations"

2. Scenario cards should show:
   - Emoji (large, centered at top of card)
   - Scenario name (bold)
```

```
   - Phrase count: "6 phrases"
   - A subtle completion indicator: once the user has played at least
     one audio from a scenario, show a small green dot.
     Track this in AsyncStorage with key "visited-{scenarioId}".

3. Card interaction:
   - Slight scale-down animation on press (pressable feedback)
   - Navigate to scenario/[id] on tap

4. The grid should be 2 columns with equal-width cards
5. Add SafeAreaView padding for iPhone notch
6. Warm background color (#FAFAF8) for the entire screen

Make sure scrolling is smooth and the layout looks good on different
iPhone sizes (test in Expo Go on your phone).
```

Afternoon Block (6 hours)

Step 3: Build the Settings screen (2 hours)

Prompt for Claude Code:

```
Build the Settings screen at app/settings.tsx.

This screen is accessed from a gear icon in the home screen header.
Add the gear icon to the home screen header now.

Settings screen contents (simple grouped list):
- Section: "App"
  - "About" → shows a small modal/alert with app version and one-line description
- Section: "Legal"
  - "Privacy Policy" → opens URL in browser (use placeholder URL for now)
  - "Terms of Use" → opens URL in browser (use placeholder URL for now)
- Section: "Feedback"
  - "Send Feedback" → opens email compose with pre-filled to address
    Use Linking.openURL('mailto:feedback@example.com?subject=App Feedback')
  - "Rate this App" → placeholder for now (comment: add StoreKit link post-launch
- Footer text: "Version 1.0.0 • Made with ❤️ in Vietnam"

Use a clean list style with chevron indicators for tappable rows.
Background: #FAFAF8. Group headers: uppercase, small, gray.
```

Step 4: Navigation polish (2 hours)

Prompt for Claude Code:

```
Review and polish the entire navigation flow:

1. Tab bar:
   - Should use custom icons (use @expo/vector-icons — Ionicons)
   - "Learn" tab: home-outline / home
   - "Saved" tab: heart-outline / heart
   - Active: #FF6B35, inactive: #9CA3AF
   - Hide tab bar labels or make them small (10px)
   - Tab bar background: white with subtle top border

2. Scenario screen header:
   - Show scenario emoji + name as title
   - Back button returns to home
   - Clean header with no shadow on iOS

3. Settings:
   - Presented as a modal (slides up from bottom) OR pushed screen
   - Has its own back/close button

4. Screen transitions should feel smooth and native
5. Status bar: dark text on light backgrounds

Test the complete navigation flow:
Home → Tap scenario → See phrases → Play audio → Save phrase →
Go back → Tap Saved tab → See saved phrase → Play audio →
Tap Learn tab → Back to home → Tap gear → Settings → Back
```

Step 5: Offline verification (1 hour)

Turn off WiFi AND cellular data on your iPhone. Test:

- App opens and shows home screen
- All scenario cards load
- Tapping a scenario shows phrases
- Audio plays for every phrase
- Favorites screen shows saved phrases
- Audio plays on favorites screen
- Nothing crashes

This should all work because everything is bundled. If anything breaks, fix it.

Step 6: Git commit (15 min)

Prompt for Claude Code:

```
Commit all changes with message: "Day 4: Favorites, settings, home polish, offlin
Push to GitHub.
```

✅ Day 4 Success Criteria

- Favorites screen shows saved phrases with “from: [Scenario]” labels
- Empty state displays when no phrases saved
- Unsaving from favorites screen removes the card with animation
- Home screen cards have press animation and visited indicators
- Settings screen works with all links
- Complete navigation flow works smoothly
- ENTIRE APP WORKS OFFLINE (WiFi and cellular off)
- All committed to GitHub

DAY 5: Content Completion + Visual Polish + Real-World Test

Hours: 12 | Objective: App feels “finished” for all 10 scenarios

Morning Block (6 hours)

Step 1: Verify all 10 scenarios are complete (2 hours)

Go through every single scenario in the app. For each one:

- All phrases display correctly with Vietnamese text, romanized, and English
- All audio files play
- No typos in English translations
- Context notes are helpful
- Card layout doesn’t break with long Vietnamese text

Prompt for Claude Code (for any issues):

```
In scenario [SCENARIO_NAME], the phrase "[VIETNAMESE TEXT]" has these issues:
[DESCRIBE ISSUE]
```

```
Fix it in content/scenarios.ts and regenerate the audio if the Vietnamese text ch
```

Step 2: Add scenario detail/intro (2 hours)

Prompt for Claude Code:

```
Add a "scenario header" section at the top of each scenario's phrase list screen.

The header should show:
- Large emoji (48px)
- Scenario name (title size)
- Description (one line, gray)
- A brief "Pro tip" relevant to that scenario in a subtle callout box
  For example:
  - Coffee Shop: "Vietnamese coffee is usually very strong. 'Cà phê sữa đá' (iced
  - Street Food: "Point at what you want if the words don't come — vendors are us
  - Grab/Taxi: "Show your destination on Google Maps if pronunciation fails."

Add these tips to each scenario in scenarios.ts as a new field: tip: string

Here are the tips for all 10 scenarios:
[I'll generate these in Claude Chat first, then paste here]
```

First, generate the tips in Claude Chat:

```
Generate a one-sentence practical travel tip for each of these 10 Vietnam
travel scenarios. The tips should be practical advice that a first-time
visitor would find useful, not language advice:

1. Coffee Shop
2. Street Food / Restaurant
3. Grab / Taxi
4. Asking the Price
5. Polite Basics
6. Convenience Store
7. Hotel / Hostel
8. Directions
9. Simple Problems
10. Small Talk

Keep each tip under 25 words. Make them genuinely useful, not generic.
```

Step 3: Visual polish pass (2 hours)

Prompt for Claude Code:

```
Do a visual polish pass on the entire app. Check and fix:

1. Consistent spacing:
   - Screen padding: px-5 (20px) everywhere
   - Card gaps: space-y-3 (12px)
   - Section spacing: mb-6 between sections

2. Typography consistency:
   - All Vietnamese text uses our vietnamese style (text-2xl font-bold text-textP
   - All romanized text uses our romanized style (text-lg font-medium text-primar
   - All English text uses our english style (text-sm text-textSecondary)

3. Loading states:
   - When navigating to a scenario, show a brief skeleton/placeholder

4. Safe areas:
   - iPhone notch is handled at the top
   - iPhone home indicator is handled at the bottom
   - No content hidden under the tab bar

5. Scroll behavior:
   - Phrase lists should bounce at the end (iOS native feel)
   - Scroll indicator should be visible

6. Touch feedback:
   - All tappable elements have a visible pressed state
   - Audio play button: opacity change on press
   - Save button: scale + color change animation
   - Scenario cards: slight scale-down on press

Review each screen on the phone and fix anything that looks off.
```

Afternoon Block (6 hours)

Step 4: REAL-WORLD FIELD TEST (3 hours)

This is the most important afternoon of your entire project. Leave your apartment. Go to these places with your app open:

1. **A coffee shop** — Use the Coffee Shop scenario. Can you find the right phrase? Is the audio loud enough? Can the barista understand the audio if you play it for them?
2. **A street food vendor** — Use the Street Food scenario. Same questions.
3. **A Grab ride** — Open the Grab/Taxi scenario during a ride. Is it useful?

For each test, write down in your BUILD_LOG.md:

- What phrase did you actually need?
- Was the phrase in the app?
- Did the audio volume feel right?
- Was the phone screen readable in sunlight?
- Did you feel silly/natural using it?
- What phrase was MISSING that you wished you had?

Step 5: Fix everything from the field test (2 hours)

Based on your notes, fix the app. Common fixes:

- Add missing phrases
- Fix romanized pronunciations that didn't help
- Adjust text sizes if unreadable in sunlight
- Increase play button size if hard to tap
- Regenerate audio for any changed phrases

Step 6: Git commit (15 min)

```
Commit: "Day 5: All 10 scenarios complete, visual polish, real-world tested"
```

✅ Day 5 Success Criteria

- All 10 scenarios have complete, polished phrase cards
- Tips/pro-tips display at the top of each scenario
- Visual consistency across all screens
- Real-world field test completed at 2+ locations
- Field test issues fixed
- App looks good enough that you'd show it to a stranger

DAY 6-7: Second Content Pass + Haptics + Animations + Edge Cases

Hours: 24 (2 days) | Objective: App feels professional and handles everything

Day 6 Morning: Content refinement

Step 1: Second native speaker review (2 hours)

Pay a different Vietnamese person (not the same one from Day 2) to go through all phrases. Fresh eyes catch different problems.

Step 2: Add "Quick Phrases" feature to home screen (3 hours)

Prompt for Claude Code:

```
Add a "Quick Phrases" section at the top of the home screen, above the scenario g

This is a horizontal scrollable row of the 5 most essential Vietnamese phrases:
- Xin chào (Hello)
- Cảm ơn (Thank you)
- Xin lỗi (Sorry/Excuse me)
- Bao nhiêu? (How much?)
- Không (No)

Each quick phrase is a small pill/chip:
- Rounded-full, orange border, white background
- Vietnamese text + small play button
- Tapping plays the audio immediately
- These are always visible on the home screen for instant access

Think of it as "the 5 words you need RIGHT NOW" without entering any scenario.
Add the section header: "Essential Phrases" with a small caption: "Tap to hear"
```

Day 6 Afternoon: Polish

Step 3: Add haptic feedback (1 hour)

Prompt for Claude Code:

```
Add haptic feedback using expo-haptics (install it).

- Light haptic on: tapping play button, tapping save button
- Medium haptic on: saving a phrase (the positive action)
- Selection haptic on: tapping a scenario card

Import and use Haptics.impactAsync() and Haptics.selectionAsync().
Only trigger haptics — don't change any other behavior.
```

Step 4: Add smooth animations (2 hours)

Prompt for Claude Code:

```
Add these animations using react-native-reanimated (already installed):

1. Scenario cards: fade-in staggered animation when home screen loads
   (each card fades in 50ms after the previous one)

2. Phrase cards: slide-up animation when scenario screen opens

3. Save button: heart "pop" animation (scale up then back) when saving

4. Audio play button: subtle pulse while audio is playing

5. Tab switching: smooth cross-fade between Learn and Saved tabs

Keep animations subtle and fast (200-300ms). The app should feel snappy,
not slow. No animations longer than 400ms.
```

Day 7: Edge Cases + Error States

Step 5: Handle every edge case (4 hours)

Prompt for Claude Code:

```
Audit and fix every edge case in the app:

1. What happens if a user has no saved phrases and goes to Saved tab?
   → Should show empty state (verify this works)

2. What happens if an audio file is missing or corrupted?
   → Show a small error icon on the play button, not a crash

3. What happens if AsyncStorage fails?
   → Wrap all AsyncStorage calls in try/catch. Log errors but don't crash.

4. What happens with very long Vietnamese phrases that wrap to 3 lines?
   → Verify layout doesn't break. Test with the longest phrase we have.

5. What happens when the user rapidly taps play on multiple phrases?
   → Only the last-tapped phrase should play. No overlapping audio.

6. What happens if the app is killed and reopened?
   → All favorites should persist. Visited indicators should persist.
```

```
7. What happens on different iPhone sizes?
   → Test layout on iPhone SE (small), iPhone 15 (standard), iPhone 15 Pro Max (l
   → In Expo Go, you can test different sizes if you have access,
     or just verify the layout uses flex properly and doesn't use fixed heights.

8. What happens when text is too long for a card?
   → Vietnamese text should wrap, not truncate
   → English text can truncate with "..." if needed

Fix ALL of these. Don't just check — actually trigger each condition and verify.
```

Step 6: Performance check (1 hour)

Prompt for Claude Code:

```
Check for performance issues:

1. Are we loading all audio files at app start? (We shouldn't be —
   only load when play is tapped)
2. Are FlatLists using proper keyExtractor and avoiding inline functions?
3. Are we re-rendering the entire phrase list when one save button is toggled?
4. Is the home screen re-rendering when switching back from Saved tab?

Fix any performance issues. Use React.memo() on PhraseCard if it's
re-rendering unnecessarily. Use useCallback for event handlers passed as props.
```

Step 7: Accessibility basics (1 hour)

Prompt for Claude Code:

```
Add basic accessibility to the app:

1. All tappable elements should have accessibilityLabel
   - Play button: "Play pronunciation for [english meaning]"
   - Save button: "Save phrase" / "Unsave phrase"
   - Scenario cards: "[scenario name], [phrase count] phrases"

2. Audio player should announce when playing:
   accessibilityHint="Double-tap to play audio"

3. Images/emojis should have accessibilityRole="image"

4. Test with VoiceOver briefly (Settings → Accessibility → VoiceOver on iPhone)
```

✅ Day 6-7 Success Criteria

- Quick Phrases section on home screen works
- Haptic feedback on all interactive elements
- Smooth animations (not janky or laggy)
- All edge cases handled (empty states, errors, rapid taps, etc.)
- Second native speaker review completed
- No crashes on any screen or interaction
- Performance is smooth (no lag when scrolling or switching tabs)
- Basic accessibility labels added

DAY 8-9: App Store Assets + Landing Page + Legal Docs

Hours: 24 (2 days) | Objective: Everything ready for App Store submission

Day 8: App Icon, Screenshots, Marketing

Step 1: Create app icon (2 hours)

In Higgsfield AI, generate icon concepts:

```
Create a minimal, modern mobile app icon. Warm orange speech bubble
on a cream/off-white background. Inside the speech bubble, subtle
Vietnamese-inspired design element (not a flag). No text, no faces.
Clean geometric style. Rounded corners (Apple will apply the mask).
Square format, 1024x1024 pixels.
```

Generate 5-10 variations. Pick the best one.

Then in Claude Code:

```
I have my app icon image at [path].

Configure it in app.json/app.config.ts as the app icon.
Expo will handle generating all the required sizes.

Also set:
- splash screen: same background color (#FAFAF8) with the icon centered
- iOS status bar style: dark
```

Step 2: Take App Store screenshots (3 hours)

You need screenshots for 6.7″ (iPhone 15 Pro Max) and 6.1″ (iPhone 15) displays.

Method: Use the iOS Simulator for pixel-perfect screenshots.

Prompt for Claude Code:

```
Help me set up for taking App Store screenshots:

1. How do I run the app in the iOS Simulator instead of Expo Go?
   (I need to do an EAS development build for this)
2. What are the exact screenshot dimensions Apple requires?
3. Walk me through taking screenshots in the Simulator

We need 4-6 screenshots showing:
1. Home screen with all 10 scenarios visible
2. Coffee Shop scenario with phrase cards
3. A phrase card with audio playing (show the playing state)
4. Favorites screen with saved phrases
5. Close-up of a phrase card showing Vietnamese, romanized, and English
```

If you can't get the Simulator working, take screenshots directly on your iPhone via Expo Go (hold Power + Volume Up).

Step 3: Create screenshot frames (2 hours)

In Higgsfield AI, create promotional framing for screenshots:

```
Create an iPhone 15 Pro mockup frame with a clean white/cream background.
Add a short marketing headline above the phone:
Screenshot 1: "10 Real Situations. One App."
Screenshot 2: "Hear it. Learn it. Use it."
Screenshot 3: "Save phrases for offline use"
```

Alternatively, use a free tool like Screenshots.pro or Previewed.app.

Step 4: Write App Store listing (1 hour)

Prompt for Claude Chat:

```
Write the complete App Store listing for my Vietnamese travel phrasebook app.

App name options (pick the best one or suggest alternatives):
- "Xin Chào: Vietnam Travel Phrases"
- "SpeakViet: Travel Vietnamese"
```

```
- "VietPhrases: Travel Vietnamese"

The app: 10 travel scenarios, 60+ phrases with native audio,
romanized pronunciation guides, save to favorites, works offline.

Write:
1. App name (max 30 chars)
2. Subtitle (max 30 chars)
3. Promotional text (max 170 chars — this can be updated without review)
4. Description (max 4000 chars)
5. Keywords (max 100 chars, comma-separated for ASO)
6. Category: Education or Travel?
7. "What's New" for version 1.0

Make the description scannable. Lead with the benefit.
Include a list of the 10 scenarios. End with a call to action.
```

Day 9: Legal Docs + EAS Build Setup

Step 5: Privacy Policy and Terms (2 hours)

Prompt for Claude Chat:

```
Write a privacy policy for my Vietnamese travel phrasebook iPhone app.

Facts:
- The app does NOT collect any personal data
- The app does NOT require an account or login
- The app does NOT send any data to any server
- All content is bundled in the app and works offline
- We use AsyncStorage (on-device only) to save favorite phrases
- No analytics, no tracking, no third-party SDKs (for V1)
- No microphone access, no camera, no location
- The developer is a U.S. individual
- The app is for adults (but no age-restricted content)

Also write Terms of Use.

Both should be:
- Plain English, not legalese
- Under 500 words each
- Cover what Apple requires
```

Step 6: Host the legal docs (1 hour)

Option A (easiest): Use Carrd

- Go to carrd.co → Sign up (free)
- Create a one-page site with sections for Privacy Policy and Terms
- Publish it → You get a URL like yourapp.carrd.co

Option B: Use GitHub Pages (free)

Prompt for Claude Code:

```
Create a simple static website in a /docs folder for GitHub Pages.

Include:
1. index.html — landing page with:
   - App name + tagline
   - 3-sentence description
   - 2 screenshot images
   - "Coming soon to the App Store" or TestFlight link
   - Email signup (just a mailto: link for now, we'll add a form later)
   - Links to Privacy Policy and Terms

2. privacy.html — Privacy Policy (full text)
3. terms.html — Terms of Use (full text)

Style: clean, minimal, mobile-friendly. Use the same orange color scheme.
No frameworks — just HTML + CSS.
```

Then enable GitHub Pages: GitHub repo → Settings → Pages → Source: main branch, /docs folder.

Step 7: Set up EAS Build (3 hours)

This is the hardest part of the entire process for a first-time developer. Take it slow.

Prompt for Claude Code:

```
Help me set up EAS Build for my Expo project step by step.

I need to:
1. Configure app.json / app.config.ts with:
   - bundle identifier (like com.yourname.viettravelphrases)
   - version: "1.0.0"
   - iOS buildNumber: "1"
   - All required iOS config

2. Set up eas.json with build profiles:
   - "development": for local testing
```

```
    - "preview": for TestFlight internal testing
    - "production": for App Store submission

3. Create my first build: eas build --platform ios --profile preview

Walk me through the entire process including:
- Apple Developer certificate setup
- Provisioning profile setup
- How to handle Apple's 2FA during the build process
- What to do when it asks about managed vs custom credentials

I am a complete beginner. Explain every prompt and option.
This is my first time doing this and I have never dealt with iOS signing before.
```

What WILL go wrong:

- Apple credentials: EAS will ask you to log in to your Apple Developer account. It may ask about 2FA codes. Follow the prompts carefully.
- Provisioning profiles: Let EAS handle these automatically ("managed credentials"). Don't try to do it manually.
- First build WILL take 15-30 minutes. Don't panic.
- If the build fails, copy the ENTIRE error log and paste it to Claude Code.

Step 8: Install TestFlight build on your phone (1 hour)

After the build succeeds:

1. Open the link from the EAS build output
2. It will show up in TestFlight app on your phone
3. Install and test the BUILT VERSION (not Expo Go) — make sure everything works identically

✅ Day 8-9 Success Criteria

- App icon created and configured
- 4-6 App Store screenshots taken and framed
- App Store listing copy written
- Privacy Policy and Terms of Use written and hosted at a URL
- Landing page live (even if minimal)

- First successful EAS Build completed
- App installed and tested via TestFlight on your physical iPhone
- All committed to GitHub

DAY 10-11: Testing Round + Bug Fixes + Friends Alpha

Hours: 24 (2 days) | Objective: 5+ people testing, all bugs fixed

Day 10: Your comprehensive test + bug fix marathon

Step 1: Complete test checklist (4 hours)

Test EVERY item on this list on the TESTFLIGHT BUILD (not Expo Go):

```
HOME SCREEN:
[ ] All 10 scenario cards display correctly
[ ] Emojis render on all cards
[ ] Phrase counts are accurate
[ ] Quick Phrases section works
[ ] Quick Phrases audio plays
[ ] Tapping a card navigates to scenario
[ ] Gear icon opens settings
[ ] Tab bar switches between Learn and Saved
[ ] Visited indicators appear after viewing a scenario

SCENARIO SCREENS (test at least 3 different scenarios):
[ ] Header shows emoji, name, description, tip
[ ] All phrase cards display Vietnamese with correct diacritical marks
[ ] All romanized pronunciation displays
[ ] All English meanings display
[ ] Audio plays for every phrase
[ ] Audio stops when playing a new phrase
[ ] Save button works (toggles heart)
[ ] Back navigation works
[ ] Scrolling is smooth with many phrases

FAVORITES:
[ ] Saved phrases appear on Favorites tab
[ ] "From: [Scenario]" label shows correctly
[ ] Audio plays on saved phrases
[ ] Unsaving removes the card
[ ] Empty state shows when no phrases saved
[ ] Re-saving a phrase makes it reappear
```

```
OFFLINE:
[ ] Turn off WiFi AND cellular
[ ] App opens
[ ] All scenarios load
[ ] All audio plays
[ ] Favorites work

EDGE CASES:
[ ] Kill the app and reopen — favorites still there
[ ] Rotate to landscape and back — no crash (lock orientation!)
[ ] Double-tap play rapidly — no crash
[ ] Save all phrases in a scenario, then unsave all
[ ] Navigate quickly between screens

SETTINGS:
[ ] Privacy Policy link opens correctly
[ ] Terms link opens correctly
[ ] Send Feedback opens email
[ ] Version number displays
```

Step 2: Fix every bug (4 hours)

For each bug, tell Claude Code the exact reproduction steps and the behavior you expected vs. what happened.

Step 3: Lock screen orientation (30 min)

Prompt for Claude Code:

```
Lock the app to portrait orientation only.
We don't support landscape. Update app.json to enforce this.
```

Step 4: Add the splash screen (1 hour)

Prompt for Claude Code:

```
Configure a proper splash screen:
- Background color: #FAFAF8
- Our app icon centered
- Use expo-splash-screen to control when it hides
- Hide the splash screen after the home screen data is ready
  (which is immediate since our data is bundled, but add
  a minimum 1-second display time so it doesn't flash)
```

Day 11: Friends Alpha

Step 5: Invite 5-10 testers to TestFlight (1 hour)

In App Store Connect → TestFlight → Internal Testing:

- Add testers by email (up to 100 for internal testing)
- They'll get an email invitation to download via TestFlight

Step 6: Watch 2-3 people use it (3 hours)

Sit with friends in Da Nang. Hand them your phone or have them install TestFlight.

DO NOT explain how the app works. Just say: "This is a Vietnamese phrases app for tourists. Try it."

Watch silently. Write down:

- Where they tap first
- Where they hesitate
- Where they look confused
- What they say out loud
- Whether they find the play button
- Whether they try to save phrases
- How long they spend on each screen
- What they say when they're done

Step 7: Collect written feedback (1 hour)

Send each tester a simple form (Google Forms or just a text message):

1. What did you like most?
2. What was confusing?
3. Would you use this on a trip to Vietnam?
4. What's missing?
5. Rate it 1-10.

Step 8: Fix top 3 issues from feedback (3 hours)

Whatever the top 3 most-mentioned issues are → fix them today.

✅ Day 10-11 Success Criteria

- Full test checklist passed
- Screen orientation locked to portrait
- Splash screen working
- 5+ external testers invited to TestFlight
- 2-3 in-person observation sessions completed
- Top 3 feedback issues fixed
- All committed and a new TestFlight build deployed

DAY 12-14: App Store Submission + Fixes + Launch Prep

Hours: 36 (3 days) | Objective: App submitted and approved

Day 12: Submit to App Store

Step 1: App Store Connect setup (2 hours)

Go to https://appstoreconnect.apple.com:

- Create a new app
- Fill in all metadata:
  - App name, subtitle, description (from Day 8)
  - Screenshots (upload for 6.7″ and 6.1″)
  - Keywords
  - Category (Education → Subcategory: Language)
  - Privacy Policy URL
  - Age rating questionnaire (rate it — no objectionable content)
  - App pricing: Free (for now)

Step 2: Production build (2 hours)

Prompt for Claude Code:

```
Create a production build for App Store submission:
```

```
eas build --platform ios --profile production

After the build completes, submit it:
eas submit --platform ios

Walk me through any prompts. This is my first submission.
```

Step 3: Submit for review (1 hour)

In App Store Connect:

- Select your build
- Add review notes: "This is a phrasebook app with pre-recorded Vietnamese audio for English-speaking travelers. No account required. All content is bundled in the app."
- Submit for review

What to expect: 24-48 hours for review. Possibly a rejection.

Day 13-14: Handle Rejection + Iterate

Common first-submission rejections and how to fix them:

Rejection Reason	Fix
Missing Privacy Policy link in the app	Add it to Settings screen + App Store Connect
"We noticed your app provides a limited user experience"	Add more content or improve the onboarding to show value faster
"Please update your app to explain why the microphone permission is required"	You shouldn't need microphone (V1 has no recording). Make sure no library is requesting it
Screenshots don't match the actual app	Retake screenshots from the production build
Crash during review	Check Sentry for crashes. Test on a clean install.
"App name or subtitle contains keyword stuffing"	Simplify the name/subtitle

If rejected, fix the issue and resubmit. Each review cycle is 24-48 hours.

While waiting for review, use these days for:

Step 4: Set up minimal analytics (2 hours)

Prompt for Claude Code:

```
Add basic analytics tracking using expo-analytics or a lightweight
custom solution with AsyncStorage.

Track these 5 events only:
1. app_opened — when the app launches
2. scenario_viewed — which scenario, with scenario_id
3. phrase_audio_played — which phrase, with phrase_id
4. phrase_saved — which phrase, with phrase_id
5. phrase_unsaved — which phrase, with phrase_id

Store events as an array in AsyncStorage under key "analytics_events".
Each event: { type, data, timestamp }

I'll manually export these from testers' devices for now.
No server needed. Just local logging.

Also create a hidden "developer" screen (accessible by tapping
the version number in Settings 5 times) that shows:
- Total events count
- Most played phrases (top 5)
- Most saved phrases (top 5)
- Button to export events as JSON (copy to clipboard)
```

Step 5: Prepare OpenClaw task organization (2 hours)

Set up your ongoing operations in OpenClaw:

```
PROJECT: Vietnamese Travel Phrases App

DAILY TASKS:
- Check Sentry for crashes (5 min)
- Check TestFlight feedback (5 min)
- Play-test 1 scenario (10 min)

WEEKLY TASKS:
- Review analytics from developer screen
- Ask 1 new person to test
- Fix top bug
- Check App Store review status

POST-LAUNCH TASKS:
- Monitor first reviews
```

```
- Respond to feedback
- Plan V1.5 features based on data

PROMPT LIBRARY:
[Store all the prompts from this guide here for reuse]
```

✅ Day 12-14 Success Criteria

- App submitted to App Store review
- All rejection issues fixed and resubmitted (if applicable)
- Basic analytics tracking in place
- OpenClaw organized with ongoing task lists
- Landing page updated with “Coming to App Store” or actual link

---

DAY 15-16: POST-SUBMISSION — V1.5 PLANNING + CONTENT EXPANSION

Hours: 24 | While waiting for Apple review

Don’t just sit around waiting. Use this time productively.

Step 1: Plan V1.5 (4 hours)

Based on all your testing and feedback, decide what V1.5 should include. Likely candidates:

- 5 more scenarios (market shopping, beach, motorbike rental, pharmacy, bar/restaurant)
- Push notifications (phrase of the day)
- The AI pronunciation practice feature (now as a premium upgrade)
- Basic account system for cloud sync
- More detailed progress tracking

Step 2: Generate content for 5 additional scenarios (8 hours)

Using the same Claude Chat workflow from Day 2, generate 5 more scenarios:

- Market / Shopping
- Beach / Swimming

- Motorbike Rental
- Pharmacy / Feeling Sick
- Bar / Nightlife

Don't add these to V1. Store them in a /content/v1.5/ folder.

Step 3: Research competitors (4 hours)

Download and use these apps for 30 minutes each:

- Ling (Vietnamese)
- Drops (Vietnamese)
- Pimsleur Vietnamese
- Any "Vietnamese travel phrases" apps on the App Store

Write down:

- What they do better than your app
- What they do worse
- What they charge
- How they onboard new users
- What their reviews say

Step 4: User outreach (4 hours)

Find communities of travelers to Vietnam:

- Reddit: r/VietNam, r/backpacking, r/digitalnomad
- Facebook groups: "Expats in Da Nang", "Vietnam Travel Tips"
- Don't promote yet — just observe what language questions people ask

Step 5: Plan monetization (4 hours)

Based on what you've learned:

- How many scenarios should be free? (Recommendation: 3-4)
- What's the monthly price that feels right? (Recommendation: $3.99-4.99/mo)
- Should you offer a one-time purchase for the Vietnamese pack? (Probably yes)

- When should you implement this? (After 50+ real users and 10+ reviews)

DAY 17-20: POST-LAUNCH — ITERATE BASED ON REAL DATA

Hours: 48 (4 days) | Objective: App live, 20+ users, data-driven improvements

Day 17: Launch day (or review resubmission)

- App approved and live on App Store
- Share with friends, family, fellow travelers
- Post in 2-3 communities (r/VietNam, Da Nang expat groups)
- Update landing page with App Store link
- Monitor Sentry for any crashes

Day 18-19: First user data

- Check downloads in App Store Connect
- Review any crash reports
- Check the developer analytics screen on a tester's phone
- Read any App Store reviews
- Identify the #1 most requested improvement
- Implement it

Day 20: Week 3 retrospective

Prompt for Claude Chat:

```
Here's my 3-week retrospective on building my Vietnamese travel phrase app:

Downloads: [NUMBER]
Active users: [NUMBER]
Most played scenario: [WHICH]
Most saved phrases: [WHICH]
Crash reports: [NUMBER]
User feedback themes: [LIST]
My observations: [LIST]
```

```
Based on this data:
1. What should I build next?
2. What should I ignore?
3. Is this product working?
4. When should I add monetization?
5. What's the single most important thing to focus on for the next 2 weeks?
```

QUICK REFERENCE: WHAT TO DO WHEN THINGS GO WRONG

Problem	What to tell Claude Code
"Red error screen on my phone"	Paste the EXACT error message. Include which screen you were on and what you tapped.
"The app won't load in Expo Go"	"Expo Go shows [error]. My Mac is running `npx expo start` and shows [status]. My phone and Mac are on the same WiFi network [name]."
"NativeWind styles aren't showing"	"Run `npx expo start --clear` to clear Metro cache. If still broken, show me metro.config.js, babel.config.js, and global.css."
"Audio doesn't play"	"I'm on [Expo Go / TestFlight build]. The play button [does/doesn't] respond to taps. The console shows [any logs]. Is `playsInSilentMode: true` set?"
"Vietnamese characters look wrong"	"The file encoding is wrong. Check that scenarios.ts is saved as UTF-8. Show me the raw bytes of the Vietnamese text."
"EAS Build failed"	Copy the ENTIRE build log and paste it. Say "This is my first build, help me fix this."
"App Store rejection"	Paste Apple's EXACT rejection message. Say "How do I fix this for resubmission?"
"I'm stuck and don't know what to do"	"Here's what I was trying to do: [X]. Here's what happened: [Y]. Here's the current state of my code: [run `git status` and `git diff`]. What should I do?"

POST-GUIDE: WHAT TO TELL OPENCLAW

When you set up OpenClaw to manage this project, give it this context:

```
I am building a Vietnamese travel phrasebook iPhone app. Here is my build plan:
[Paste the day-by-day summary]

Your job:
1. Create a daily task list based on which day I'm on
2. Remind me of the success criteria for each day
3. Track which days I've completed
4. Flag if I'm spending too long on any step (suggest moving on)
5. Keep a running list of bugs/issues/feedback
6. Remind me to commit to git at the end of each day
7. Remind me to write in BUILD_LOG.md at the end of each day

If I ask about a feature that's not in the V1 scope, tell me
"That's a V1.5 feature. Add it to the backlog and keep shipping V1."
```

Total estimated calendar time: 20 working days at 12 hours/day Total estimated actual time with problems: 22-25 days Total estimated additional cost: $100-150 (Apple Developer + API credits + content review)